COSC 39: Theory of Computation

Credit Statement: Talked to Carson Bates'27, and Sophie Park'27.

Problem 1. Let N be a given NFA $N = (Q, S, A, \Sigma, \delta)$. The language accepted by N is defined as

$$L(N) = \{ w \in \Sigma^* : \delta^*(S, w) \cap A \neq \varnothing \}.$$

Prove that the following language associated to N is also automatic.

$$L^{\subseteq}(N) = \{ w \in \Sigma^* : \delta^*(S, w) \subseteq A \}.$$

(Now we can take intersection of two automatic languages with ease!)

Solution.

From a theorem proved in class we know that for any given NFA N, there exists a DFA D recognizing the same language. This DFA has the following construction:

- $Q_D = \mathcal{P}(Q_N)$, where $\mathcal{P}(Q_N)$ is the power set of Q_N.
- $S_D = S_N$ where $S_N = \{s_i \mid s_i \text{ is a starting state in } N\} \in \mathcal{P}(Q_N)$.
- $A_D = \{X \in Q_D \mid X \cap A_N \neq \emptyset\}$.
- $\partial_D(P, a) = \partial_N(P, a)$.
- Also note that for any $w \in \Sigma^\star$ we have $\partial_N^\star(S, w) = \partial_D^\star(S, w)$.

To show that $L^{\subseteq}(N)$ is an automatic language we take this DFA and create a new DFA D' by only editing the accepting set as follows,

$$A_{D'} = \{X \in Q_D \mid X \subseteq A_N\}$$

Now, for any $w \in L(D')$ we have the following by construction,

$$\partial_D^\star(S, w) \in A_{D'} \iff \partial_N^\star(S, w) \subseteq A_N \iff w \in L^{\subseteq}(N)$$

Since D' is a DFA, $L(D') = L^{\subseteq}(N)$ is an automatic language.

Problem 2. For a string $w \in \Sigma^*$, let $\operatorname{rev}(w)$ denote w written backwards (or ε if $w = \varepsilon$). Consider an arbitrary automatic language L recognized by some DFA D. Prove that the following language is also automatic.

$$\operatorname{rev}(L) = \{\operatorname{rev}(x) : x \in L\}.$$

In other words, instead of reading the input from first to last symbol, one can read the input backwards from the last symbol to the first.

Solution.

We construct a NFA N as follows:

- We added a new start state called s_N with ε-transitions to all $a \in A_D$ where A_D were the accepting states in D.
- $Q_N = Q_D \cup \{s_N\}$.
- $A_N = \{s_D\}$, where s_D is the start state for D.
- For every transition $\delta_D(q, a) = q'$ in D we add a transition $\delta_N(q', a) = q$ in N.

To see that $w \in L \iff \operatorname{rev}(w) \in L(N)$ consider the following.

$(\implies)$

Assume $\delta_D^\star(s_D, w) \in A_D$, where $w = a_1 \cdots a_n$. Then there exists a path,

$$s_D \xrightarrow{a_1} q_1 \xrightarrow{a_2} \cdots \xrightarrow{a_n} q_n$$

where $q_n \in A_D$. Now consider $\delta_N^\star(s_N, \operatorname{rev}(w))$, where $\operatorname{rev}(w) = a_n \cdots a_1$. By the reversed-transition construction we have a path,

$$s_N \xrightarrow{\varepsilon} q_n \xrightarrow{a_n} \cdots \xrightarrow{a_2} q_1 \xrightarrow{a_1} s_D$$

Since $s_D \in A_N$ we have $\delta_N^\star(s_N, \operatorname{rev}(w)) \in A_N$.

$(\impliedby)$

Assume $\delta_N^\star(s_N, w) \in A_N$ where $w = a_1 \cdots a_n$, then there exists an accepting path in N:

$$s_N \xrightarrow{\varepsilon} q_n \xrightarrow{a_1} \cdots \xrightarrow{a_{n-1}} q_1 \xrightarrow{a_n} s_D$$

with $q_n \in A_D$. By the reversed-transition construction, this gives us a path in D:

$$s_D \xrightarrow{a_n} q_{n-1} \xrightarrow{a_{n-1}} \cdots \xrightarrow{a_1} q_n \in A_D$$

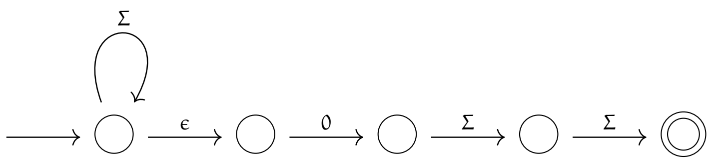
so $\delta_D^\star(s_D, \operatorname{rev}(w)) \in A_D$.

Hence $w \in L(N) \iff \operatorname{rev}(w) \in L$. And we have $L(N) = \operatorname{rev}(L)$. Since every NFA has an equivalent DFA, $\operatorname{rev}(L)$ is automatic.

Problem 3. Construct a DFA for the language that contains all binary strings where the third-to-last symbol is 0. Give an English explanation of what each state represents.

Solution.

Since we showed in class that there exists a DFA equivalent to every NFA and a NFA equivalent to every ϵ-DFAs, there exists some DFA equivalent to the following ϵ-DFA construction.



The ε-DFA loops at the start state with any symbol in the alphabet, and with every new symbol read we ε-transition to begin checking whether the next symbol is 0 followed by exactly two more symbols. Hence, we only accept strings if the third-to-last symbol is 0. If the input is shorter than 3 inputs we enter a fail state.